

INTERNATIONAL CYBERSECURITY AND DIGITAL FORENSICS ACADEMY

LABORATORY INVESTIGATION & FORENSIC EVIDENCE REPORT

“Preparing Global Cybersecurity Leaders Through Excellence in Hands-On Training”

Lab Title: SBT-DF203 Lab 6: Firewall Traffic Control and Forensic Verification

Course Name: Basic Networking Skills for Digital Forensics

Student Name: Almustapha Yusuf

Registration Number: fwsd2511343

Operating Platform: Kali Linux 2026 (Rolling Release)

Date of Submission: 18/09/2026

Investigation Scope: Host-Based Packet Filtering (iptables) on Linux

Report Version: Version 1.0 (Final Release)

Executive Summary

This report documents the forensic validation of a Linux host-based firewall (iptables) controlling HTTP traffic. The original ruleset was preserved and hashed. A baseline HTTP session was captured and verified as allowed. A source-specific DROP rule was then inserted into the INPUT chain, and the blocked connection attempt was captured — showing SYN retransmissions and timeout behaviour. Critically, the iptables counter for the DROP rule incremented to 7 packets / 420 bytes, providing definitive corroboration that the rule matched the blocked traffic. The rule was then safely removed and access was restored.

1. Lab Network and Scope

- Host Server:** Kali Linux (192.168.199.128/24) running Apache2 on TCP port 80.
- Default Gateway:** 192.168.199.2 (Layer 2 MAC: 00:50:56:ff:50:a0).
- Blocked Client (Simulated):** Same interface IP (192.168.199.128) — documented as a single-VM lab environment limitation.
- Capture Interfaces:** eth0 (interface-level telemetry) and lo (local loopback capture).

2. Original Firewall State and Evidence Hashes

Prior to making any packet filtering modifications, the active kernel netfilter ruleset was exported and preserved:

```
Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
num  pkts bytes target    prot opt in     out     source         destination

Chain FORWARD (policy ACCEPT 0 packets, 0 bytes)

Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)
```

Evidence Artifact	SHA-256 Hash Digest	Status
iptables_before.rules	e3b0c44298fc1c149afb4c8996fb92427ae41e4649b934ca495991b7852b855	EMPTY RULESET
http_allowed.pcapng	24ec745abb9c570777bc1c2e3672729b6e0132c6a7a6762b3c71a5312f1524ed2	VERIFIED

3. Allowed Baseline Analysis

- **Baseline Loopback Query:** curl -v http://127.0.0.1/firewall_lab.html returned HTTP 200 OK (119 bytes payload displaying 'Server: Almustapha Yusuf').
- **Baseline Interface Query:** curl -v http://192.168.199.128/firewall_lab.html returned HTTP 200 OK (119 bytes).
- **Packet Capture Verification:** Captured 10 packets on interface lo for tcp port 80. Dissection confirmed full compliant RFC 793 sequence: SYN -> SYN-ACK -> ACK -> HTTP GET -> HTTP 200 OK -> FIN teardown.

4. Rule Implementation and Verification

A source-specific packet dropping rule was inserted at the head of the INPUT netfilter chain:

```
sudo iptables -I INPUT 1 -s 192.168.199.128 -p tcp --dport 80 -j DROP
```

Inspection Output: Kernel Ruleset Verification:

```
Chain INPUT (policy ACCEPT)
num  pkts bytes target    prot opt in  out  source          destination
1     0     0  DROP      tcp  --  *   *   192.168.199.128 0.0.0.0/0      tcp dpt:80
```

- **Rule Check Affirmation:** The presence of the rule was confirmed programmatically via sudo iptables -C INPUT ... returning code 0 ('Rule verified present.').

5. Blocked Traffic Analysis and Counter Correlation

- **Client Behavioral Timeout:** curl -v --connect-timeout 10 http://192.168.199.128/firewall_lab.html aborted with error: curl: (28) Connection timed out after 10002 milliseconds.
- **Packet Capture Telemetry:** Dissection of http_blocked.pcapng revealed 7 consecutive SYN attempts (relative sequence 0) spaced across TCP exponential backoff intervals with 0 SYN-ACK responses and 0 HTTP payload bytes.
- **Hardware/Kernel Counter Corroboration:** Post-test inspection of iptables -L INPUT -n -v showed rule 1 incremented to 7 packets and 420 bytes (7 frames x 60 bytes IPv4/TCP header).
- **Forensic Correlation:** 7 SYN frames captured by TShark = exactly 7 packets registered by the iptables DROP counter. This mathematically corroborates that the firewall rule was the direct causal agent blocking the connection.

6. Allowed vs Blocked Comparison

Comparative matrix contrasting network and forensic indicators between allowed baseline and blocked traffic:

Investigative Indicator	Allowed Baseline Capture	Blocked Test Capture
Client SYN visible?	YES	YES
Server SYN-ACK visible?	YES	NO

Handshake completed?	YES	NO
HTTP GET visible?	YES	NO
HTTP response visible?	YES	NO
Retransmissions / timeouts?	NO	YES (7 SYN attempts)
iptables counter change?	0 packets (rule absent)	7 packets, 420 bytes
cURL execution result	HTTP 200 OK	Connection timed out

7. DROP vs REJECT Discussion

- **DROP Target Action:** Silently discards packets at the kernel netfilter hook without sending an ICMP or TCP response. The connecting client remains ignorant of network reachability and repeatedly retransmits SYN packets following exponential retry backoffs until hitting client connection timeout thresholds. This matches the exact behavior captured in this lab.
- **REJECT Target Action:** Explicitly terminates connection attempts by transmitting an active response back to the sender — typically an ICMP Destination Unreachable (Port Unreachable) or a TCP segment with the RST/ACK flag asserted. The client fails fast with an immediate 'Connection refused' notification without issuing retransmissions.

8. Cleanup and Restored Service

To ensure operational continuity, the DROP rule was removed by exact specification:

```
sudo iptables -D INPUT -s 192.168.199.128 -p tcp --dport 80 -j DROP
```

- **Rule Absence Verification:** Executed `sudo iptables -C INPUT ...` which failed with 'Bad rule (does a matching rule exist in that chain?)', confirming rule unlinking.
- **Service Restoration Test:** `curl -v --connect-timeout 5 http://192.168.199.128/firewall_lab.html` immediately succeeded, returning HTTP/1.1 200 OK.
- **Process Audit:** Executed `ps aux | grep -E '[t]shark'` verifying zero background packet sniffing processes remained active.

9. Forensic Interpretation Questions

- **Why does DROP cause retransmissions and timeouts?:** DROP silently discards incoming packets without notifying the client. Because the client never receives a SYN-ACK, TCP retransmission algorithms assume the packet was lost in transit and retry transmission until the connect timeout expires.
- **How would REJECT differ in the packet trace?:** With REJECT, the firewall actively returns an ICMP Port Unreachable packet or a TCP RST segment. The client immediately aborts with 'Connection refused', leaving zero retransmission frames in the capture.
- **Why are iptables packet counters forensically valuable?:** Counters provide independent, kernel-level corroboration. Demonstrating that a specific firewall rule counted exactly 7 packets (420 bytes) proves causality: the traffic was dropped specifically because of the configured rule, not because of routing errors or server downtime.
- **How do you distinguish a dead web service from a firewall block?:** If the web daemon is stopped but no firewall rule exists, the operating system kernel will respond to a SYN packet with a TCP RST segment. If a

firewall DROP rule is present, the kernel stays completely silent, resulting in repeated client SYN retries and timeout.

- **What is the operational risk of deleting rules by line number?:** In dynamic production environments, rules inserted or deleted by other processes change the line numbering of the chain. Deleting by line number risks purging an unintended rule. Deleting by explicit rule specification guarantees precision.

10. Screenshots and Evidence

The following 13 figures document the step-by-step firewall rule implementation, baseline allowed traffic, blocked timeout captures, counter corroborations, and service restoration:

```
(kali@kali)~[~/SBT-DF203-Lab5]
$ cd ~
mkdir -p ~/SBT-DF203-Lab6/{evidence,working,exported,reports,screenshots,scripts}
cd ~/SBT-DF203-Lab6

(kali@kali)~[~/SBT-DF203-Lab6]
$ pwd
.
$ find . -maxdepth 1 -type d -print
/home/kali/SBT-DF203-Lab6
.
./exported
./evidence
./working
./screenshots
./scripts
./reports

(kali@kali)~[~/SBT-DF203-Lab6]
$ sudo apt update
sudo apt install -y apache2 curl iptables tshark wireshark
[sudo] password for kali:
Get:1 http://kali.download/kali kali-rolling InRelease [34.0 kB]
Get:2 http://kali.download/kali kali-rolling/main amd64 Packages [21.6 MB]
Get:3 http://kali.download/kali kali-rolling/main amd64 Contents (deb) [53.9 MB]
Get:4 http://kali.download/kali kali-rolling/contrib amd64 Packages [116 kB]
Get:5 http://kali.download/kali kali-rolling/contrib amd64 Contents (deb) [270 kB]
Get:6 http://kali.download/kali kali-rolling/non-free amd64 Packages [183 kB]
Get:7 http://kali.download/kali kali-rolling/non-free amd64 Contents (deb) [915 kB]
Fetched 76.9 MB in 36s (2,157 kB/s)
2081 packages can be upgraded. Run 'apt list --upgradable' to see them.
curl is already the newest version (8.21.0-2).
tshark is already the newest version (4.6.6-1).
wireshark is already the newest version (4.6.6-1).
The following packages were automatically installed and are no longer required:
gir1.2-girepository-2.0 libcrypt-dev libkmlbase1t64 libmplex2-2.1-0t64 libqt6test6 python3-fs rpcsvc-proto
libann0 libgav1-1 libkmlcore1t64 libpoppler147 libqt6xml6 python3-gpg samba-ad-dc
libarmadillo14 libgdal37 libkmlengine1t64 libportmidi0 libraw23t64 python3-pyqt6.sip samba-ad-provision
libavc1394-0 libgirepository-1.0-1 liblab-gamut1 libpython3.13-dev libwireshark18 python3.13 samba-dsdb-modules
libblosc2-4 libpgmep6t64 libmbedcrypto16 libqt6openglwidgets6 libwiretap15 python3.13-dev
libcdt5 libjs-jquery-ui libmjpegutils-2.1-0t64 libqt6sql6 libwesutil16 python3.13-minimal
libcgraph6 libjs-underscore libmpeg2encpp-2.1-0t64 libqt6sql6-sqlite libzxing3 python3.13-venv
```

Figure 1: *Workspace Initialization & Package Installation* — Creating /SBT-DF203-Lab6 hierarchy and installing Apache2, cURL, iptables, and TShark.

```

kali@kali: ~/SBT-DF203-Lab6
$ sudo systemctl enable --now apache2
$ sudo systemctl status apache2 --no-pager
systemctl status apache2 --no-pager
Synchronizing state of apache2.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable apache2
● apache2.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: disabled)
   Active: active (running) since Thu 2026-09-17 16:56:20 EDT; 28s ago
     Invocation: a1b8d4fee742466cb209c2ff16a110bf
       Docs: https://httpd.apache.org/docs/2.4/
    Main PID: 773935 (apache2)
      Status: "Total requests: 0; Idle/Busy workers 100/0; Requests/sec: 0; Bytes served/sec: 0 B/sec"
        Tasks: 6 (limit: 2197)
       Memory: 13.6M (peak: 14M)
          CPU: 146ms
    CGroup: /system.slice/apache2.service
            └─773935 /usr/sbin/apache2 -k start -DFOREGROUND
              └─773939 /usr/sbin/apache2 -k start -DFOREGROUND
                └─773940 /usr/sbin/apache2 -k start -DFOREGROUND
                  └─773941 /usr/sbin/apache2 -k start -DFOREGROUND
                    └─773943 /usr/sbin/apache2 -k start -DFOREGROUND
                      └─773944 /usr/sbin/apache2 -k start -DFOREGROUND

Sep 17 16:56:20 kali systemd[1]: Starting apache2.service - The Apache HTTP Server...
Sep 17 16:56:20 kali apache2[773935]: AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using 127.0.1.1...his message
Sep 17 16:56:20 kali systemd[1]: Started apache2.service - The Apache HTTP Server.
Hint: Some lines were ellipsized, use -l to show in full.

(kali@kali)~/(~/SBT-DF203-Lab6)
$ printf '<DOCTYPE html>\n<html><body><h1>ICDFA Network Forensics Firewall Lab</h1><p>Server: Almustapha Yusuf</p></body></html>\n' | sudo tee /var/www/html/firewall_lab.html
cat /var/www/html/firewall_lab.html
<DOCTYPE html>
<html><body><h1>ICDFA Network Forensics Firewall Lab</h1><p>Server: Almustapha Yusuf</p></body></html>
<DOCTYPE html>
<html><body><h1>ICDFA Network Forensics Firewall Lab</h1><p>Server: Almustapha Yusuf</p></body></html>

(kali@kali)~/(~/SBT-DF203-Lab6)
$ cd ~/SBT-DF203-Lab6
$ sudo iptables-save | tee reports/iptables_before.rules

```

Figure 2: Apache Service Start, HTML Creation & Ruleset Baseline — Starting apache2, generating /var/www/html/firewall_lab.html with student credentials, and saving initial ruleset.

```

(kali@kali)~/(~/SBT-DF203-Lab6)
$ cd ~/SBT-DF203-Lab6
$ sudo iptables-save | tee reports/iptables_before.rules
$ sha256sum reports/iptables_before.rules | tee reports/iptables_before_sha256.txt
$ cat reports/iptables_before.txt
$ cat reports/iptables_before_sha256.txt
Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
num  pkts bytes target     prot opt in     out     source               destination
Chain FORWARD (policy ACCEPT 0 packets, 0 bytes)
num  pkts bytes target     prot opt in     out     source               destination
Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)
num  pkts bytes target     prot opt in     out     source               destination
e3b0c44298fc1c149afb4c8996fb92427ae41e4649b934ca495991b7852b855  reports/iptables_before.rules
Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
num  pkts bytes target     prot opt in     out     source               destination
Chain FORWARD (policy ACCEPT 0 packets, 0 bytes)
num  pkts bytes target     prot opt in     out     source               destination
Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)
num  pkts bytes target     prot opt in     out     source               destination
e3b0c44298fc1c149afb4c8996fb92427ae41e4649b934ca495991b7852b855  reports/iptables_before.rules

(kali@kali)~/(~/SBT-DF203-Lab6)
$ ip -br address | tee reports/server_interfaces.txt
ip route | tee reports/server_routes.txt
$ sudo ss -ltnp | grep ':80' | tee reports/apache_listener.txt
$ cat reports/server_interfaces.txt
$ cat reports/server_routes.txt
$ cat reports/apache_listener.txt
lo        UNKNOWN    127.0.0.1/8  ::1/128
eth0      UP          192.168.199.128/24 fe80::8c84:5c9c:8758:3178/64
default via 192.168.199.2 dev eth0 proto dhcp src 192.168.199.128 metric 100
192.168.199.0/24 dev eth0 proto kernel scope link src 192.168.199.128 metric 100
LISTEN 0      511          *:80        *:         users:((("apache2",pid=773944,fd=4),("apache2",pid=773943,fd=4),("apache2",pid=773941,fd=4),("apach
e2",pid=773940,fd=4),("apache2",pid=773939,fd=4),("apache2",pid=773935,fd=4)))

```

Figure 3: Baseline Ruleset Verification & Interface Inventory — Displaying empty initial iptables rules, SHA-256 hash calculation, network IP configuration, and Apache listener verification.

```

(kali@kali)-[~/SBT-DF203-Lab6]
$ cd ~/SBT-DF203-Lab6
curl -v --connect-timeout 5 http://127.0.0.1/firewall_lab.html 2>&1 | tee reports/baseline_curl.txt
cat reports/baseline_curl.txt
* Trying 127.0.0.1:80 ...
* Established connection to 127.0.0.1 (127.0.0.1 port 80) from 127.0.0.1 port 39932
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload  Total   Spent    Left   Speed
  0     0    0     0    0     0     0      0      0  0* using HTTP/1.x
> GET /firewall_lab.html HTTP/1.1
> Host: 127.0.0.1
> User-Agent: curl/8.21.0
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 200 OK
< Date: Thu, 17 Sep 2026 21:11:52 GMT
< Server: Apache/2.4.68 (Debian)
< Last-Modified: Thu, 17 Sep 2026 20:57:35 GMT
< ETag: "77-65bb408870fce"
< Accept-Ranges: bytes
< Content-Length: 119
< Vary: Accept-Encoding
< Content-Type: text/html
<
{ [119 bytes data]
100  119 100   119    0     0  5608     0      0
* Connection #0 to host 127.0.0.1:80 left intact
<!DOCTYPE html>
<html><body><h1>ICDFA Network Forensics Firewall Lab</h1><p>Server: Almustapha Yusuf</p></body></html>
* Trying 127.0.0.1:80 ...
* Established connection to 127.0.0.1 (127.0.0.1 port 80) from 127.0.0.1 port 39932
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload  Total   Spent    Left   Speed
  0     0    0     0    0     0     0      0      0  0* using HTTP/1.x
> GET /firewall_lab.html HTTP/1.1
> Host: 127.0.0.1
> User-Agent: curl/8.21.0
> Accept: */*

```

Figure 4: Baseline Loopback HTTP Query — Executing curl on localhost (127.0.0.1) returning HTTP 200 OK (119 bytes).

```
kali@kali: ~/SBT-DF203-Lab6  kali@kali: ~/SBT-DF203-Lab5
0 0 0 0 0 0 0 0 0* using HTTP/1.x
> GET /firewall_lab.html HTTP/1.1
> Host: 127.0.0.1
> User-Agent: curl/8.21.0
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 200 OK
< Date: Thu, 17 Sep 2026 21:11:52 GMT
< Server: Apache/2.4.68 (Debian)
< Last-Modified: Thu, 17 Sep 2026 20:57:35 GMT
< ETag: "77-65bb408870fce"
< Accept-Ranges: bytes
< Content-Length: 119
< Vary: Accept-Encoding
< Content-Type: text/html
<
{ [119 bytes data]
100 119 100 119 0 0 5608 0
* Connection #0 to host 127.0.0.1:80 left intact
<!DOCTYPE html>
<html><body><h1>ICDFA Network Forensics Firewall Lab</h1><p>Server: Almustapha Yusuf</p></body></html>

(kali@kali)-[~/SBT-DF203-Lab6]
$ cd ~/SBT-DF203-Lab6
IFACE=lo
tshark -i $IFACE -f 'tcp port 80' -w evidence/http_allowed.pcapng
Warning: program compiled against libxml 215 using older 214
Capturing on 'Loopback: lo'
10 ^C

(kali@kali)-[~/SBT-DF203-Lab6]
$ cd ~/SBT-DF203-Lab6
cp --preserve=timestamp evidence/http_allowed.pcapng working/http_allowed_working.pcapng
sha256sum evidence/http_allowed.pcapng | tee reports/http_allowed_sha256.txt
cat reports/http_allowed_sha256.txt
24ec745abb9c570777b1c2e3672729b6e0132c6a7a6762b3c71a5312f1524ed2 evidence/http_allowed.pcapng
24ec745abb9c570777b1c2e3672729b6e0132c6a7a6762b3c71a5312f1524ed2 evidence/http_allowed.pcapng

(kali@kali)-[~/SBT-DF203-Lab6]
$ █
```

Figure 5: Allowed HTTP Traffic Capture & Hash Preservation — TShark capture of permitted HTTP session (http_allowed.pcapng) and computing SHA-256 digest.

```

(kali@kali)-[~/SBT-DF203-Lab6]
$ cd ~/SBT-DF203-Lab6
curl -v --connect-timeout 5 http://192.168.199.128/firewall_lab.html 2>&1 | tee reports/baseline_iface_curl.txt
cat reports/baseline_iface_curl.txt
* Trying 192.168.199.128:80 ...
* Established connection to 192.168.199.128 (192.168.199.128 port 80) from 192.168.199.128 port 44596
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload  Total   Spent    Left   Speed
  0     0    0     0    0     0      0      0      0  0* using HTTP/1.x
> GET /firewall_lab.html HTTP/1.1
> Host: 192.168.199.128
> User-Agent: curl/8.21.0
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 200 OK
< Date: Thu, 17 Sep 2026 21:30:30 GMT
< Server: Apache/2.4.68 (Debian)
< Last-Modified: Thu, 17 Sep 2026 20:57:35 GMT
< ETag: "77-65bb408870fce"
< Accept-Ranges: bytes
< Content-Length: 119
< Vary: Accept-Encoding
< Content-Type: text/html
<
{ [119 bytes data]
100  119 100   119    0     0  2141    0
* Connection #0 to host 192.168.199.128:80 left intact
<!DOCTYPE html>
<html><body><h1>ICDFA Network Forensics Firewall Lab</h1><p>Server: Almustapha Yusuf</p></body></html>
* Trying 192.168.199.128:80 ...
* Established connection to 192.168.199.128 (192.168.199.128 port 80) from 192.168.199.128 port 44596
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload  Total   Spent    Left   Speed
  0     0    0     0    0     0      0      0      0  0* using HTTP/1.x
> GET /firewall_lab.html HTTP/1.1
> Host: 192.168.199.128
> User-Agent: curl/8.21.0
> Accept: */*

```

Figure 6: Baseline Interface IP HTTP Query — Executing curl against interface IP (192.168.199.128) returning HTTP 200 OK prior to firewall rule insertion.

```

kali@kali: ~/SBT-DF203-Lab6
Session Actions Edit View Help
kali@kali: ~/SBT-DF203-Lab6
> User-Agent: curl/8.21.0
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 200 OK
< Date: Thu, 17 Sep 2026 21:30:30 GMT
< Server: Apache/2.4.68 (Debian)
< Last-Modified: Thu, 17 Sep 2026 20:57:35 GMT
< ETag: "77-65bb408870fce"
< Accept-Ranges: bytes
< Content-Length: 119
< Vary: Accept-Encoding
< Content-Type: text/html
<
{ [119 bytes data]
100  119 100   119    0     0  2141    0
* Connection #0 to host 192.168.199.128:80 left intact
<!DOCTYPE html>
<html><body><h1>ICDFA Network Forensics Firewall Lab</h1><p>Server: Almustapha Yusuf</p></body></html>

(kali@kali)-[~/SBT-DF203-Lab6]
$ BLOCKED_CLIENT_IP=192.168.199.128
sudo iptables -I INPUT 1 -s "$BLOCKED_CLIENT_IP" -p tcp --dport 80 -j DROP
sudo iptables -L INPUT -n -v --line-numbers | tee reports/iptables_after_add.txt
cat reports/iptables_after_add.txt
[sudo] password for kali:
Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
num  pkts bytes target    prot opt in     out     source            destination
1     0     0 DROP      tcp  --  *      *        192.168.199.128   0.0.0.0/0          tcp dpt:80
Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
num  pkts bytes target    prot opt in     out     source            destination
1     0     0 DROP      tcp  --  *      *        192.168.199.128   0.0.0.0/0          tcp dpt:80

(kali@kali)-[~/SBT-DF203-Lab6]
$ sudo iptables -C INPUT -s "$BLOCKED_CLIENT_IP" -p tcp --dport 80 -j DROP 66 printf 'Rule verified present.\n' | tee reports/rule_verification.txt
cat reports/rule_verification.txt
Rule verified present.
Rule verified present.

(kali@kali)-[~/SBT-DF203-Lab6]
$

```

Figure 7: Firewall DROP Rule Insertion & Rule Check — Inserting DROP rule for source 192.168.199.128 on port 80 and validating with iptables -C.

```

(kali@kali)-[~/SBT-DF203-Lab5]
$ cd ~/SBT-DF203-Lab6
curl -v --connect-timeout 10 http://192.168.199.128/firewall_lab.html 2>&1 | tee reports/blocked_curl.txt
cat reports/blocked_curl.txt
* Trying 192.168.199.128:80 ...
* Connection timed out after 10002 milliseconds
* closing connection #0
curl: (28) Connection timed out after 10002 milliseconds
* Trying 192.168.199.128:80 ...
* Connection timed out after 10002 milliseconds
* closing connection #0
curl: (28) Connection timed out after 10002 milliseconds

(kali@kali)-[~/SBT-DF203-Lab6]
$

```

Figure 8: Blocked Traffic Client Timeout Trace — Executing curl against 192.168.199.128 resulting in connection timeout after 10002 ms.

```

(kali@kali)-[~/SBT-DF203-Lab6]
$ cd ~/SBT-DF203-Lab6
IFACE=lo
tshark -i $IFACE -f 'tcp port 80' -w evidence/http_blocked.pcapng
Warning: program compiled against libxml 215 using older 214
Capturing on 'Loopback: lo'
7 ^C

(kali@kali)-[~/SBT-DF203-Lab6]
$ cd ~/SBT-DF203-Lab6
sudo iptables -L INPUT -n -v --line-numbers | tee reports/iptables_after_test.txt
cat reports/iptables_after_test.txt
[sudo] password for kali:
Chain INPUT (policy ACCEPT 293 packets, 30234 bytes)
num  pkts  bytes  target    prot opt in     out     source               destination
1      7    420 DROP      tcp  --  *      *        192.168.199.128      0.0.0.0/0           tcp dpt:80
Chain INPUT (policy ACCEPT 293 packets, 30234 bytes)
num  pkts  bytes  target    prot opt in     out     source               destination
1      7    420 DROP      tcp  --  *      *        192.168.199.128      0.0.0.0/0           tcp dpt:80

(kali@kali)-[~/SBT-DF203-Lab6]
$ cp --preserve=timestamp evidence/http_blocked.pcapng working/http_blocked_working.pcapng
sha256sum evidence/http_blocked.pcapng | tee reports/http_blocked_sha256.txt
cat reports/http_blocked_sha256.txt
30812e81789c21ae3f8768b99f433e1af538d7b693d374ba5d95ed7c65f78610  evidence/http_blocked.pcapng
30812e81789c21ae3f8768b99f433e1af538d7b693d374ba5d95ed7c65f78610  evidence/http_blocked.pcapng

(kali@kali)-[~/SBT-DF203-Lab6]
$

```

Figure 9: Blocked Capture & Counter Corroboration (7 pkts / 420 bytes) — Capturing blocked traffic (http_blocked.pcapng), hashing capture file, and proving iptables rule counter incremented to 7 packets / 420 bytes.

```

(kali@kali) [~/SBT-DF203-Lab6]
$ cd ~/SBT-DF203-Lab6
echo "===== ALLOWED CAPTURE =====" | tee reports/allowed_vs_blocked.tsv
tshark -r working/http_allowed_working.pcapng -V 'tcp.flags.syn==1 || http.request || http.response || tcp.analysis.retransmission' -T fields -e frame.number -e frame.time.relative -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport -e tcp.flags -e tcp.analysis.retransmission -e http.request.uri -e http.response.code 2>/dev/null | tee -a reports/allowed_vs_blocked.tsv
echo "" | tee -a reports/allowed_vs_blocked.tsv
echo "===== BLOCKED CAPTURE =====" | tee -a reports/allowed_vs_blocked.tsv
tshark -r working/http_blocked_working.pcapng -V 'tcp.flags.syn==1 || http.request || http.response || tcp.analysis.retransmission' -T fields -e frame.number -e frame.time.relative -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport -e tcp.flags -e tcp.analysis.retransmission -e http.request.uri -e http.response.code 2>/dev/null | tee -a reports/allowed_vs_blocked.tsv
cat reports/allowed_vs_blocked.tsv
===== ALLOWED CAPTURE =====
1 0.000000000 127.0.0.1 40342 127.0.0.1 80 0*0002
2 0.000017229 127.0.0.1 80 127.0.0.1 40342 0*0012
4 0.000694721 127.0.0.1 40342 127.0.0.1 80 0*0018 /firewall_lab.html
6 0.003783026 127.0.0.1 80 127.0.0.1 40342 0*0018 /firewall_lab.html 200

===== BLOCKED CAPTURE =====
1 0.000000000 192.168.199.128 42398 192.168.199.128 80 0*0002
2 1.014729247 192.168.199.128 42398 192.168.199.128 80 0*0002 1
3 2.038070926 192.168.199.128 42398 192.168.199.128 80 0*0002 1
4 3.061912815 192.168.199.128 42398 192.168.199.128 80 0*0002 1
5 4.085976244 192.168.199.128 42398 192.168.199.128 80 0*0002 1
6 5.110809081 192.168.199.128 42398 192.168.199.128 80 0*0002 1
7 7.126220022 192.168.199.128 42398 192.168.199.128 80 0*0002 1

===== ALLOWED CAPTURE =====
1 0.000000000 127.0.0.1 40342 127.0.0.1 80 0*0002
2 0.000017229 127.0.0.1 80 127.0.0.1 40342 0*0012
4 0.000694721 127.0.0.1 40342 127.0.0.1 80 0*0018 /firewall_lab.html
6 0.003783026 127.0.0.1 80 127.0.0.1 40342 0*0018 /firewall_lab.html 200

===== BLOCKED CAPTURE =====
1 0.000000000 192.168.199.128 42398 192.168.199.128 80 0*0002
2 1.014729247 192.168.199.128 42398 192.168.199.128 80 0*0002 1
3 2.038070926 192.168.199.128 42398 192.168.199.128 80 0*0002 1
4 3.061912815 192.168.199.128 42398 192.168.199.128 80 0*0002 1
5 4.085976244 192.168.199.128 42398 192.168.199.128 80 0*0002 1
6 5.110809081 192.168.199.128 42398 192.168.199.128 80 0*0002 1
7 7.126220022 192.168.199.128 42398 192.168.199.128 80 0*0002 1

```

Figure 10: Side-by-Side Packet Stream Dissection — TShark comparison showing allowed capture (4 frames to 200 OK) vs blocked capture (7 SYN retransmissions).

```

kali@kali: ~/SBT-DF203-Lab6
(kali@kali)-[~/SBT-DF203-Lab6]
$ cat > reports/comparison_table.txt << 'EOF'
=== ALLOWED vs BLOCKED COMPARISON ===

Indicator | Allowed Capture | Blocked Capture
Client SYN visible? | YES | YES
Server SYN-ACK visible? | YES | NO
Handshake completed? | YES | NO
HTTP GET visible? | YES | NO
HTTP response visible? | YES | NO
Retransmissions/timeouts? | NO | YES
iptables counter change? | 0 packets (rule did not exist yet) | 7 packets, 420 bytes
curl result? | HTTP 200 OK | Connection timeout
EOF
cat reports/comparison_table.txt
=== ALLOWED vs BLOCKED COMPARISON ===

Indicator | Allowed Capture | Blocked Capture
Client SYN visible? | YES | YES
Server SYN-ACK visible? | YES | NO
Handshake completed? | YES | NO
HTTP GET visible? | YES | NO
HTTP response visible? | YES | NO
Retransmissions/timeouts? | NO | YES
iptables counter change? | 0 packets (rule did not exist yet) | 7 packets, 420 bytes
curl result? | HTTP 200 OK | Connection timeout

(kali@kali)-[~/SBT-DF203-Lab6]
$ cd ~/SBT-DF203-Lab6
BLOCKED_CLIENT_IP=192.168.199.128
sudo iptables -D INPUT -s "$BLOCKED_CLIENT_IP" -p tcp --dport 80 -j DROP
sudo iptables -L INPUT -n -v --line-numbers | tee reports/iptables_restored.txt
cat reports/iptables_restored.txt
Chain INPUT (policy ACCEPT 300 packets, 30772 bytes)
num pkts bytes target prot opt in out source destination
Chain INPUT (policy ACCEPT 300 packets, 30772 bytes)
num pkts bytes target prot opt in out source destination

```

Figure 11: Allowed vs Blocked Indicator Table & Rule Deletion — Reviewing comparison table and deleting DROP rule from INPUT chain via explicit specification.

```

(kali@kali)-[~/SBT-DF203-Lab6]
$ sudo iptables -C INPUT -s "$BLOCKED_CLIENT_IP" -p tcp --dport 80 -j DROP 2>&1 || echo 'Rule successfully removed' | tee reports/rule_removal_check.txt
cat reports/rule_removal_check.txt
iptables: Bad rule (does a matching rule exist in that chain?).
Rule successfully removed
Rule successfully removed

(kali@kali)-[~/SBT-DF203-Lab6]
$ curl -v --connect-timeout 5 http://192.168.199.128/firewall_lab.html 2>&1 | tee reports/restored_access.txt
cat reports/restored_access.txt | head -20
* Trying 192.168.199.128:80 ...
* Established connection to 192.168.199.128 (192.168.199.128 port 80) from 192.168.199.128 port 38038
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload  Total   Spent    Left     Speed
0      0    0     0    0     0      0      0      0  0* using HTTP/1.x
> GET /firewall_lab.html HTTP/1.1
> Host: 192.168.199.128
> User-Agent: curl/8.21.0
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 200 OK
< Date: Fri, 18 Sep 2026 18:51:06 GMT
< Server: Apache/2.4.68 (Debian)
< Last-Modified: Thu, 17 Sep 2026 20:57:35 GMT
< ETag: "77-65bb408870fce"
< Accept-Ranges: bytes
< Content-Length: 119
< Vary: Accept-Encoding
< Content-Type: text/html
<
{ [119 bytes data]
100  119 100   119    0     0  1311    0      0
* Connection #0 to host 192.168.199.128:80 left intact
<!DOCTYPE html>
<html><body><h1>ICDFA Network Forensics Firewall Lab</h1><p>Server: Almustapha Yusuf</p></body></html>
* Trying 192.168.199.128:80 ...
* Established connection to 192.168.199.128 (192.168.199.128 port 80) from 192.168.199.128 port 38038
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload  Total   Spent    Left     Speed
0      0    0     0    0     0      0      0      0  0* using HTTP/1.x

```

Figure 12: Rule Removal Verification & Restored Access — Verifying rule absence with iptables -C and testing curl returning HTTP 200 OK.

```

< Last-Modified: Thu, 17 Sep 2026 20:57:35 GMT
< ETag: "77-65bb408870fce"
< Accept-Ranges: bytes
< Content-Length: 119
< Vary: Accept-Encoding
< Content-Type: text/html
<
{ [119 bytes data]
100 119 100 119 0 0 1311 0
* Connection #0 to host 192.168.199.128:80 left intact
<!DOCTYPE html>
<html><body><h1>ICDFA Network Forensics Firewall Lab</h1><p>Server: Almustapha Yusuf</p></body></html>
* Trying 192.168.199.128:80 ...
* Established connection to 192.168.199.128 (192.168.199.128 port 80) from 192.168.199.128 port 38038
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload  Total   Spent    Left   Speed
  0      0  0      0  0      0     0      0      0      0  0* using HTTP/1.x
> GET /firewall_lab.html HTTP/1.1
> Host: 192.168.199.128
> User-Agent: curl/8.21.0
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 200 OK
< Date: Fri, 18 Sep 2026 18:51:06 GMT
< Server: Apache/2.4.68 (Debian)
< Last-Modified: Thu, 17 Sep 2026 20:57:35 GMT
< ETag: "77-65bb408870fce"
< Accept-Ranges: bytes
< Content-Length: 119
< Vary: Accept-Encoding
< Content-Type: text/html

(kali@kali)-[~/SBT-DF203-Lab6]
$ ps aux | grep -E '[t]shark' | tee reports/process_check.txt
cat reports/process_check.txt
echo "(Empty = no capture processes running, clean)"
(Empty = no capture processes running, clean)

(kali@kali)-[~/SBT-DF203-Lab6]
$

```

Figure 13: Service Restoration Confirmation & Process Cleanup — Reviewing restored HTTP response headers and confirming zero lingering TShark capture processes via ps aux.

11. Conclusions

This lab successfully demonstrated host-based firewall control and forensic verification. The DROP rule blocked HTTP traffic from the specified source, evidenced by: (1) client curl connection timeout, (2) 7 consecutive SYN retransmissions in the packet capture, and (3) an exact matching iptables counter increment (7 packets / 420 bytes). The rule was safely removed, and access was restored. All evidence was preserved with matching SHA-256 hashes.

12. Appendix – Commands and Hashes

```
# Baseline Ruleset Export
sudo iptables-save | tee reports/iptables_before.rules
sudo iptables -L -n -v --line-numbers

# Capture Allowed Traffic
tshark -i lo -f 'tcp port 80' -w evidence/http_allowed.pcapng
curl -v http://192.168.199.128/firewall_lab.html

# Insert Source DROP Rule
sudo iptables -I INPUT 1 -s 192.168.199.128 -p tcp --dport 80 -j DROP
sudo iptables -C INPUT -s 192.168.199.128 -p tcp --dport 80 -j DROP

# Capture Blocked Traffic & Inspect Counters
tshark -i lo -f 'tcp port 80' -w evidence/http_blocked.pcapng
curl -v --connect-timeout 10 http://192.168.199.128/firewall_lab.html
sudo iptables -L INPUT -n -v --line-numbers

# Remove Rule & Restore Access
sudo iptables -D INPUT -s 192.168.199.128 -p tcp --dport 80 -j DROP
curl -v --connect-timeout 5 http://192.168.199.128/firewall_lab.html
```

Evidence Integrity: Cryptographic Hash Digest Preservation:

- **Original Ruleset Hash:** e3b0c44298fc1c149afb4c8996fb92427ae41e4649b934ca495991b7852b855
- **Allowed Traffic Capture Hash:** 24ec745abb9c570777bc1c2e3672729b6e0132c6a7a6762b3c71a5312f1524ed2
- **Blocked Traffic Capture Hash:** 30812e81789c21ae3f8768b99f433e1af538d7b693d374ba5d95ed7c65f78610

Student Declaration

I confirm that I completed this lab in an authorized environment, preserved the evidence, did not target any third-party system, and accurately documented my own commands, observations, and conclusions.

Student Signature: Almustapha Yusuf

Date: 18/09/2026